

```
data = {
    'column_a': ['one', 'two', 'three',
                'four'],
    'column_b': [101, 102, 103, 104],
    'column_c': [201, 202, 203, 204],
    'column_d': [401, 402, 403, 404]
}
df1=pd.DataFrame(data)
df1
# Reading CSV/JSON
df = pd.read_csv('purchases.csv')
df = pd.read_json('purchases.json')
df

# Converting back to a CSV, JSON,
df.to_csv('new_purchases.csv')
df.to_json('new_purchases.json')

# Handling duplicates
temp_df = temp_df.drop_duplicates()
temp_df.shape
```

How to work with missing values?  
There are two options in dealing with nulls:

- Get rid of rows or columns with nulls.
- Replace nulls with non-null values, a technique known as imputation:

```
movies_df.isnull().sum()
movies_df.dropna()
```

Other than just dropping rows, you can also drop columns with null values by setting *axis=1*: *movies\_df.dropna(axis=1)*

Imputation is a conventional engineering technique used to keep valuable data that has null values. It aims to replace missing data with substituted values.

```
#Age is a column name for our train
data
mean_value=train['Age'].mean()
train['Age']=train['Age'].fillna(mean_
value)#this will replace all NaN values
train['Age']=train['Age'].fillna(median_
value)
```

One alternative to using a loop

to iterate over a DataFrame is to use *pandas.apply()*.

To print the first three rows from *emp.csv*, type:

```
import pandas as pd
from statistics import *
df = pd.read_csv("emp.csv").head(3)
df
```

| empid | name  | dept    | salary |
|-------|-------|---------|--------|
| 1     | jack  | health  | 20000  |
| 2     | rohan | science | 30000  |
| 3     | tinku | sales   | 15000  |

To convert to all caps without loop, type:

```
allcaps = lambda c: c.upper()
df['name']=df['name'].apply(allcaps)
df.head(3)
```

| empid | name  | dept    | salary |
|-------|-------|---------|--------|
| 1     | JACK  | health  | 20000  |
| 2     | ROHAN | science | 30000  |
| 3     | TINKU | sales   | 15000  |

Apply formulae to all rows as follows:

```
def sal_increment(row):
    return row['salary']+row['salary']
* 0.50
```

```
df['revised_sal']=df.apply(sal_
increment, axis=1)
df
```

| empid | name  | dept    | revised_ salary |
|-------|-------|---------|-----------------|
| 1     | JACK  | health  | 30000           |
| 2     | ROHAN | science | 45000           |
| 3     | TINKU | sales   | 22500           |

## Scikit-learn: an impressive machine learning library in Python

### Key features

- Free machine learning library for Python with inbuilt ML algorithms — vector machine, random forests and *k-neighbors*.

- Built on NumPy, SciPy and Matplotlib.
- Scikit-learn comes with a few standard data sets; for instance, the iris and digits data sets for classification and the diabetes data set for regression.

### Tricks to try out!

*Impute missing values:* Scikit-learn offers multiple ways to impute missing values. Here, we consider two approaches. The *SimpleImputer* class provides basic strategies for imputing missing values (through the mean or median for example). A more sophisticated approach is the *KNNImputer* class, which provides imputation for filling in missing values using the *K-Nearest Neighbors* approach. Each missing value is imputed using values from *n\_neighhors nearest neighbors* that have a value for the particular feature. The values of the neighbours are averaged uniformly or weighted by distance to each neighbour.

#Example of using both imputation methods:

```
from sklearn.experimental import
enable_iterative_imputer
from sklearn.impute import
SimpleImputer, KNNImputer
from sklearn.datasets import make_
classification
import pandas as pd
```

```
X, y = make_classification(n_samples=5,
n_features=4, n_classes=2, random_
state=123)
```

```
X = pd.DataFrame(X,
columns=['Feature_1', 'Feature_2',
'Feature_3', 'Feature_4'])
```

```
X.iloc[1, 2] = float('NaN')
```

```
X.iloc[1, 2] = float('NaN')
```

```
imputer_simple = SimpleImputer()
```

```
pd.DataFrame(imputer_simple.fit_
transform(X))
```

```
imputer_KNN = KNNImputer(n_neighbors=2,
weights="uniform")pd.DataFrame(imputer_
KNN.fit_transform(X))
```